



Sistema HappySite

Documentação técnica e guia de acesso

Administração de Obras — Cerqueira Engenharia

Gerado em: Agosto de 2026

1. O que é o HappySite

O HappySite é um sistema de administração de obras desenvolvido sob medida, para controle de colaboradores, empreiteiros, medições mensais e pagamentos, com um desenho esquemático visual do prédio (por andar/apartamento) para acompanhamento dos serviços executados em cada etapa da obra.

O sistema funciona **pela internet**, acessível de qualquer computador, celular ou tablet com navegador — não precisa instalar nada. Todos os dados ficam centralizados em um único banco de dados na nuvem, então qualquer alteração feita por um usuário aparece automaticamente para todos os outros.

Navegador (qualquer computador) → Render (roda o sistema) → Neon (banco de dados)

2. Como acessar o sistema (uso diário)

Link do sistema:

<https://happysite-6oi6.onrender.com>

Usuário padrão inicial	admin
Senha	definida por você no primeiro acesso (troca obrigatória)

Atenção — Plano gratuito: o servidor "dorme" após 15 minutos sem uso. Quando alguém acessa depois desse período, a primeira tela pode demorar de 30 a 50 segundos para carregar (o servidor está "acordando"). Isso é normal e não é erro. Se quiser eliminar essa espera, é possível migrar para um plano pago do Render (a partir de ~US\$ 7/mês).

3. Estrutura técnica — os 3 serviços usados

3.1. GitHub — onde fica o código-fonte

É um "cofre" na internet que guarda todo o código do sistema (frontend + backend) e o histórico de todas as alterações já feitas. Toda vez que uma alteração é enviada para lá, o Render é avisado automaticamente e atualiza o site sozinho.

Repositório:

<https://github.com/happySiteCerqueira/happysite>

Login de acesso: conta GitHub da organização/usuário `happySiteCerqueira` (login e senha configurados por você ao criar a conta).

3.2. Render — onde o sistema "roda" (hospedagem)

É o serviço que pega o código do GitHub, "liga" o sistema e o deixa disponível 24 horas por dia no link público. É aqui que ficam os logs (para diagnosticar problemas), as variáveis de ambiente (como a senha de conexão do banco de dados) e as configurações de deploy.

Painel de controle:

<https://dashboard.render.com>

Login de acesso: conta usada para criar o serviço no Render (e-mail/senha ou login via GitHub).

Nome do serviço dentro do Render: `happysite` (ou nome similar — aparece na lista principal do dashboard após login).

3.3. Neon — onde ficam os dados (banco de dados PostgreSQL)

É onde ficam armazenadas de fato todas as informações reais do sistema: obras cadastradas, colaboradores, empreiteiros, medições, valores, pagamentos,

usuários, etc. O Render se conecta a esse banco toda vez que alguém usa o sistema.

Painel de controle:

<https://console.neon.tech>

Login de acesso: conta usada para criar o banco de dados na Neon.

Nome do projeto/banco: `neondb` (região: sa-east-1 — São Paulo).

4. Resumo de acessos (preencher e guardar em local seguro)

Serviço	Link	Usuário/E-mail	Senha
Sistema (uso diário)	happysite-6oi6.onrender.com	admin	_____
GitHub	github.com	_____	_____
Render	dashboard.render.com	_____	_____
Neon	console.neon.tech	_____	_____

Recomendação: guarde essas senhas em um gerenciador de senhas ou em um documento protegido (não deixe em texto simples em qualquer lugar).

5. Como acessar e editar o projeto em outro computador (usando o Cline/IA)

O código-fonte do sistema não está "preso" a este computador — ele está salvo no GitHub (item 3.1). Isso significa que qualquer computador com acesso à internet e às credenciais corretas consegue baixar o projeto e continuar o desenvolvimento, inclusive usando uma ferramenta de IA como o Cline (a mesma usada para criar este sistema).

Passo a passo para configurar em um novo computador:

Passo 1 — Instalar os programas necessários

- [Node.js](#) (versão 18 ou superior)
- [Git](#)
- [Visual Studio Code](#)
- Extensão **Cline** instalada dentro do VS Code (na aba de Extensões)

Passo 2 — Baixar o código do projeto (clonar o repositório)

Abra o terminal (Prompt de Comando/PowerShell) na pasta onde deseja salvar o projeto e rode:

```
git clone https://github.com/happySiteCerqueira/happysite.git
```

Isso vai criar uma pasta chamada `happysite` com todo o código atualizado (a versão mais recente que está publicada no GitHub).

Passo 3 — Abrir a pasta no VS Code

Abra o Visual Studio Code, use "Arquivo → Abrir Pasta" e selecione a pasta `happysite` que acabou de ser baixada.

Passo 4 — Configurar a conexão com o banco de dados

Dentro da pasta `server`, crie um arquivo chamado `.env` (baseado no arquivo de exemplo `server/.env.example`) e cole a mesma `DATABASE_URL` do banco Neon (disponível no painel da Neon, item 3.3, em "Connection string").

Passo 5 — Instalar as dependências e testar localmente (opcional)

```
npm run install:all  
npm run dev
```

Isso liga uma cópia do sistema rodando localmente no computador, útil para testar mudanças antes de publicá-las.

Passo 6 — Abrir o Cline e pedir alterações

Na barra lateral do VS Code, clique no ícone do Cline. Ele vai automaticamente identificar que está dentro da pasta do projeto HappySite (reconhece a estrutura de arquivos, o `package.json`, o repositório Git conectado etc.) e já estará "apto" a fazer alterações, criar novas telas, corrigir bugs ou adicionar funcionalidades — exatamente como foi feito durante o desenvolvimento inicial.

Passo 7 — Publicar as alterações (deploy automático)

Depois que o Cline (ou você) fizer as alterações desejadas, basta rodar no terminal:

```
git add -A  
git commit -m "descrição da alteração feita"  
git push
```

Isso envia as mudanças para o GitHub. O Render detecta automaticamente o novo envio e atualiza o sistema publicado em poucos minutos — sem precisar fazer mais nada manualmente.

Importante: para conseguir enviar alterações (`git push`) para o repositório, o computador precisa estar autenticado com uma conta que tenha permissão de escrita no repositório GitHub `happySiteCerqueira/happysite`. Se for outro computador/pessoa, pode ser necessário fazer login no Git com a conta correta ou receber um convite de colaborador no GitHub.

6. Estrutura de pastas do projeto (referência rápida)

Pasta/Arquivo	O que é
<code>client/</code>	Frontend (telas, componentes React) que o usuário vê no navegador
<code>server/</code>	Backend (rotas da API, regras de negócio, conexão com o banco)
<code>server/db/database.js</code>	Configuração de conexão com o PostgreSQL (Neon)
<code>server/routes/</code>	Cada arquivo é um grupo de funcionalidades (obras, usuários, medições, etc.)
<code>render.yaml</code>	Configuração de como o Render deve construir e rodar o projeto
<code>iniciar.bat</code> / <code>iniciar-dev.bat</code>	Atalhos para rodar o sistema localmente no Windows

7. Contato de suporte

Para dúvidas técnicas futuras, este mesmo fluxo (abrir a pasta do projeto no VS Code + Cline) permite continuar o desenvolvimento com qualquer assistente de IA compatível, já que todo o histórico e contexto do projeto está documentado no próprio código e neste arquivo.